Introduction to Search

CSI 4106

Marcel Turcotte
Version: Nov 3, 2026 18:48

Preamble

Message of the Day

David Silver and Lasse Espeholt of Ineffable Intelligence: A Superlearner for the Age of Experience, posted on the Sequoia Capital YouTube channel on 2026-10-13.

David Silver led the DeepMind research behind AlphaGo, the first AI system to defeat a professional human Go player, and its successors, AlphaZero and MuZero. He is also a professor of computer science at University College London and a Fellow of the Royal Society. His work connects the neural-network methods introduced earlier in the course with the search methods introduced here. His personal website provides his current biography.

Learning Objectives

- **Describe** the role of search algorithms in AI, crucial for planning, reasoning, and applications like AlphaGo.
- **Formulate** a search problem using states, an initial state, goal states, actions, a transition model, and action costs.
- **Distinguish** states from search nodes, and reached states from frontier and expanded nodes.
- **Implement** BFS and DFS and **compare** them using the 8-Puzzle problem.
- **Explain** the completeness, cost optimality, and memory requirements of BFS and DFS under stated assumptions.

This presentation motivates search methods in artificial intelligence and establishes the vocabulary for the search unit. Breadth-first and depth-first search provide familiar baselines for studying how a problem is represented, how nodes are selected, and why more selective methods are needed. The unit covers the following themes.

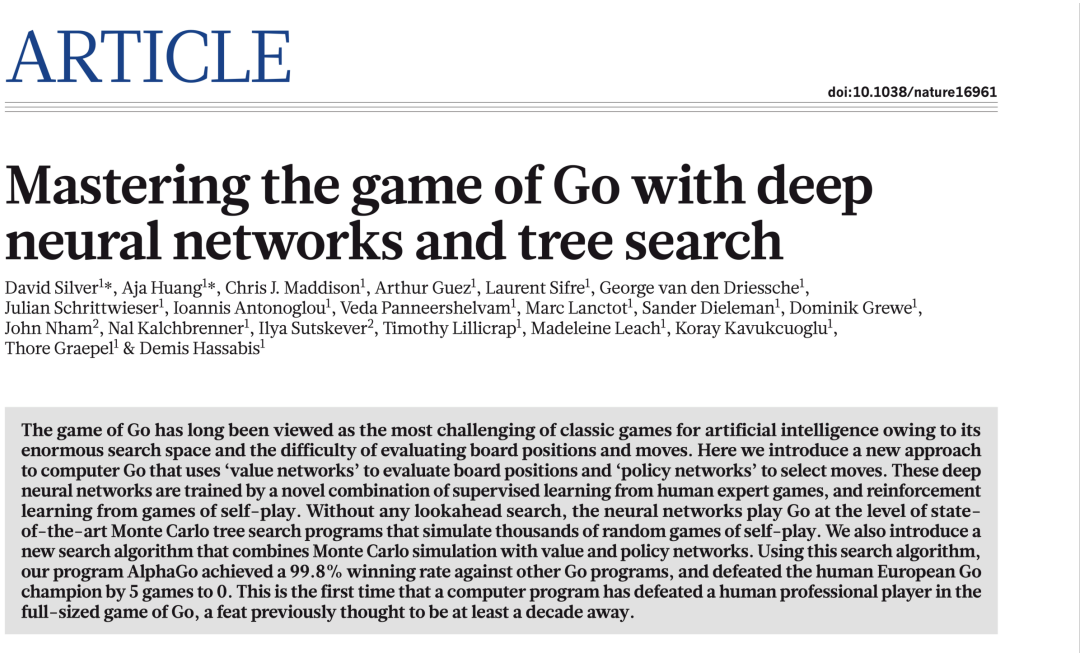
- Uninformed Search
- Informed Search

- Local Search
- Population-Based Metaheuristics
- Adversarial Search
- Monte Carlo Tree Search

These methods also provide a foundation for formal reasoning, where search can explore possible deductions or candidate solutions.

Justification

Justification



ARTICLE

doi:10.1038/nature16961

Mastering the game of Go with deep neural networks and tree search

David Silver[1]*, Aja Huang[1]*, Chris J. Maddison[1], Arthur Guez[1], Laurent Sifre[1], George van den Driessche[1], Julian Schrittwieser[1], Ioannis Antonoglou[1], Veda Panneershelvam[1], Marc Lanctot[1], Sander Dieleman[1], Dominik Grewe[1], John Nham[2], Nal Kalchbrenner[1], Ilya Sutskever[2], Timothy Lillicrap[1], Madeleine Leach[1], Koray Kavukcuoglu[1], Thore Graepel[1] & Demis Hassabis[1]

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence owing to its enormous search space and the difficulty of evaluating board positions and moves. Here we introduce a new approach to computer Go that uses 'value networks' to evaluate board positions and 'policy networks' to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte Carlo simulation with value and policy networks. Using this search algorithm, our program AlphaGo achieved a 99.8% winning rate against other Go programs, and defeated the human European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.

Silver et al. (2016)

Justification

We have honed our expertise in **machine learning** to a point where we possess a robust understanding of **neural networks** and **deep learning**, allowing us to develop simple models using **Keras**.

. . .

In recent years, **Monte Carlo Tree Search (MCTS)** has played a pivotal role in advancing **artificial intelligence** research. After initially concentrating on deep learning, **we are now shifting our focus to search**.

Justification

The integration of **deep learning** and **MCTS** underpins modern applications such as **AlphaGo**, **AlphaZero**, and **MuZero**.

. . .

Search algorithms are crucial in addressing challenges in **planning** and **reasoning** and are likely to become **increasingly significant in future developments**.

Search (a biased timeline)

- **1968 – A***: Heuristic search for minimum-cost paths.
- **1970s–1980s – Population-Based Algorithms** (e.g., Genetic Algorithms): Population-based stochastic search for optimization.
- **1980s – Constraint Satisfaction Problems (CSPs)**: Advances in combining constraint propagation with backtracking search.
- **2013 – DQN**: Learning action values directly from pixels through deep Q-learning.
- **2015 – AlphaGo**: Deep neural networks guide MCTS; first match victory against a professional Go player.
- **2017 – AlphaZero**: Self-play learning with neural-network-guided MCTS for chess, shogi, and Go.
- **2019 – MuZero**: Planning with a learned model, without being supplied the environment's dynamics.
- **2020 – Agent57**: Deep reinforcement learning with directed exploration and adaptive policy selection across 57 Atari games.
- **2023 – FunSearch**: Evolutionary search over programs proposed by an LLM and evaluated automatically.
- **2024 – AlphaGeometry**: Geometry theorem proving combining a neural language model with symbolic deduction.

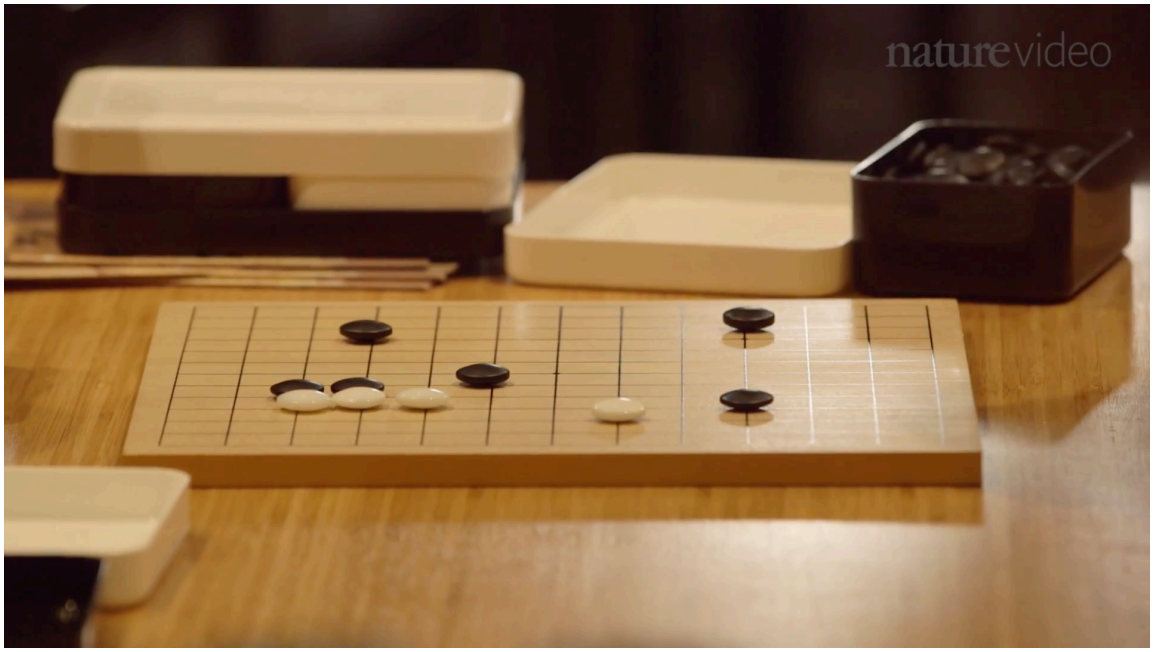
This is a selected timeline of search and related learning milestones. DQN and Agent57 learn how to act through reinforcement learning; they do not perform the explicit look-ahead tree search used by AlphaGo, AlphaZero, and MuZero. Exploration during learning and search over possible future action sequences are distinct ideas.

The dates refer to selected milestones, not necessarily journal publication dates. AlphaGo defeated Fan Hui in October 2015 before its Nature publication in January 2016. AlphaZero and MuZero first appeared as preprints in 2017 and 2019, respectively. The population-method and CSP entries identify periods of development rather than the origins of those fields.

Sources: Hart et al. (1968); Holland's 1975 book; constraint-consistency algorithms, 1985; DQN, 2013; AlphaZero, 2017; MuZero, 2019; Agent57, 2020; FunSearch, 2023;

AlphaGeometry, 2024.

Search



The computer that mastered Go. Nature Video posted on YouTube on 2016-01-27. **In-class video** (7**m** 52**s**).

Viewing question: How can learned knowledge help decide which moves to explore and how to evaluate a position?

In 1997, IBM's Deep Blue defeated the reigning world chess champion, Garry Kasparov. It could evaluate 200 million chess positions per second, combining fast computation with selective search, move ordering, and carefully designed evaluation. In a later reflection, Kasparov described it as being "as intelligent as your alarm clock." This comment contrasts task-specific performance with general intelligence; it does not mean that Deep Blue lacked sophisticated search algorithms. See IBM's history and the developers' technical account.

The Nature video explains how learned knowledge can make Go search selective: a policy network suggests promising moves, and a value network estimates the quality of positions. Search combines these estimates with exploration of possible continuations. Later, AlphaZero learned through self-play and combined learned evaluation with MCTS. Kasparov remarked, "I can't disguise my satisfaction that it plays with a very dynamic style, much like my own!" See Google DeepMind's AlphaZero article.

The key connection is that learning and search complement one another. This lecture establishes the vocabulary and baseline search methods; Lecture 22 examines MCTS in detail.

AlphaGo - The Movie

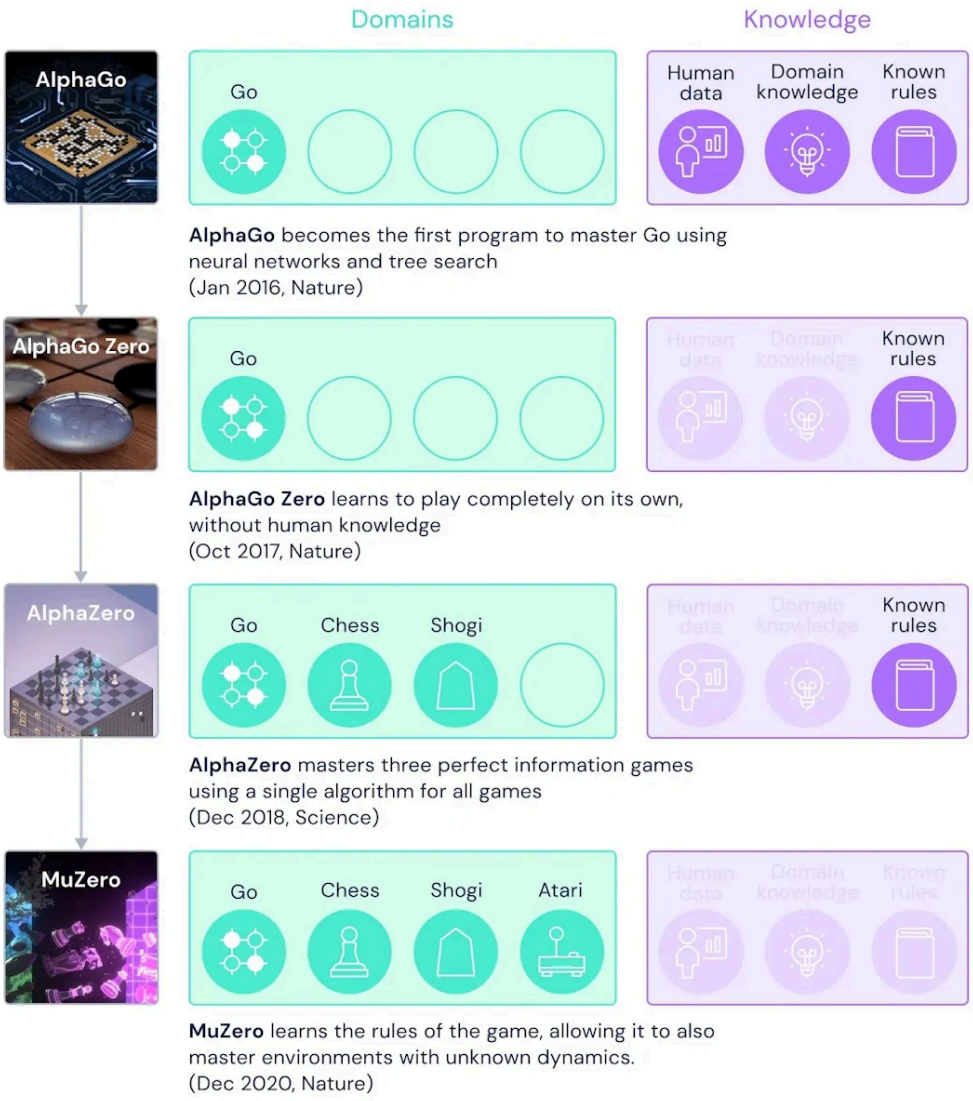


Yes, **AlphaGo** successfully defeated Go champion Lee **Sedol**, 4–1 in March 2016. The full documentary follows that match. (1**h** 30**m**)

AlphaGo - The Movie | Full award-winning documentary, Google DeepMind, 2020-03-13.

The Nature feature predates the Lee Sedol match: it introduces AlphaGo's 5–0 victory over professional player Fan Hui in October 2015. The documentary is a supplementary resource about the subsequent match. See Google DeepMind's AlphaGo history.

AlphaGo2MuZero



Attribution: MuZero: Mastering Go, chess, shogi and Atari without rules, Google DeepMind, 2020-12-23. Schrittwieser et al. (2020)

David Silver led the DeepMind research behind these projects and is a professor at University College London. He earned his PhD at the University of Alberta, where he was supervised by Richard Sutton.

You might also want to watch the following video: AlphaGo Zero: Discovering new knowledge. Posted on YouTube on 2017-10-18.

Search

Applications

- **Pathfinding and Navigation**: Used in robotics and video games to find a **path** from a **starting point** to a **destination**.

- **Puzzle Solving**: Solving puzzles like the **8-puzzle**, **mazes**, or **Sudoku**.
- **Network Analysis**: Analyzing networks and graphs, such as finding **connectivity** or **shortest paths** in **social networks** or **transportation maps**.
- **Game Playing**: Used to evaluate moves in games like **chess** or **Go**, especially when combined with other strategies.

Applications

- **Scheduling**: Planning and scheduling tasks in **manufacturing**, **project management**, or **airline scheduling**.
- **Resource Allocation**: Allocating resources in a network or within an organization where **constraints** must be satisfied.
- **Configuration Problems**: Solving problems where a set of components must be assembled to meet specific requirements, such as configuring a computer system or **designing a circuit**.

Applications

- **Decision Making under Uncertainty**: Used in **real-time strategy games** and **simulations** where decisions need to be evaluated under uncertain conditions.

- **Storytelling**: LLMs can effectively generate stories when guided by a valid input plan from an **automated planner**. (Simon and Muise 2024)

The applications of search algorithms are both **numerous** and **diverse**.

Search has been an active area of research not only because of its wide range of applications but also due to the potential for improvements in algorithms to significantly reduce program execution time or facilitate the exploration of larger search spaces.

Outline

1. **Deterministic & Heuristic Search**: BFS, DFS, A* for pathfinding and optimization in classical AI.

2. **Population-Based Algorithms**: Focus on structured problems and stochastic search.

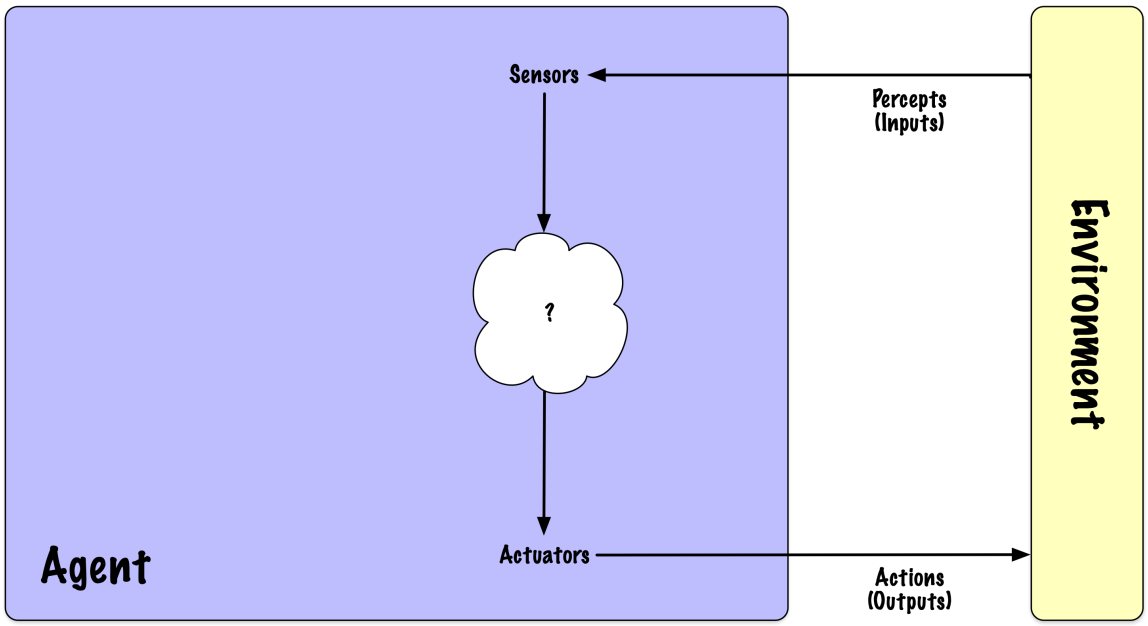
3. **Adversarial Game Algorithms**: Minimax, alpha-beta pruning, MCTS for decision-making in competitive environments.

This lecture and the upcoming ones will thoroughly cover these topics.

Definition

> **(Russell and Norvig 2020, 63)**
>
> When the correct action to take is not immediately obvious, an agent may need to **plan ahead**: to consider a **sequence of actions** that form a **path** to a **goal state**. Such an agent is called a **problem-solving agent**, and the computational process it undertakes is called **search**.

Terminology



An **agent** is an entity that performs **actions**. A **rational agent** selects actions expected to achieve the **best outcome**, given its information and performance measure. Conceptually, an **agent** perceives its environment through **sensors** and interacts with it using **actuators**.

The definition of an agent in **artificial intelligence (AI)** shares some similarities with the **psychological** definition, but there are key distinctions. In AI, an agent is an **autonomous entity** that perceives its **environment** through **sensors** and acts upon it using **actuators** to achieve specific **goals**. Both definitions involve **perception**, **decision-making**, and **action**.

However, while psychological agents are human or biological and involve **complex cognitive** and **emotional processes**, AI agents are computational and operate based on algorithms designed to **maximize certain performance measures** or **achieve predefined objectives**. The focus in AI is more on the technical implementation of these processes, whereas in psychology, the emphasis is on understanding the cognitive and motivational aspects of agency.

The concept of **agentic design** in software engineering and artificial intelligence has experienced a resurgence in popularity.

Environment Characteristics

- **Observability**: Partially observable, or *fully observable*
- **Agent Composition**: *Single* or multiple agents
- **Predictability**: *Deterministic* or non-deterministic
- **Decision Dependency**: Episodic or *sequential*
- **Temporal Dynamics**: *Static* or dynamic
- **State Representation**: *Discrete* or continuous

In this lecture, environments are assumed to be: *fully observable*, *single agent*, *sequential*, *deterministic*, *static*, and *discrete*.

The characteristics of an environment influence the complexity of problem-solving.

- A **fully observable environment** allows the agent to detect all relevant aspects for decision-making.
- In a **deterministic environment**, the agent can predict the next state based on the current state and its subsequent action.
- **Episodic environments** involve independent decisions: an action in one episode does not affect later episodes. Classifying independent images is an example.
- **Sequential environments** require considering how an action affects future decisions. The 8-puzzle and chess are examples. A state can summarize all relevant history without making the task episodic.
- A **dynamic environment** can change independently while the agent deliberates. A **static environment** does not change during deliberation, but the agent's actions can still change its state.
- Chess serves as an example of a **discrete environment**, with distinct states and actions. Discrete spaces can be finite or countably infinite. In contrast, an autonomous vehicle operates within a **continuous-state** and **continuous-time environment**.

Problem-Solving Process

Search: The agent simulates possible sequences of actions in a model to find a path to a goal state. A successful sequence is a **solution**, which the agent can then execute.

A precise formulation facilitates the development of reusable code.

Under these assumptions, a solution, when one exists, can be expressed as a fixed sequence of actions rather than a policy that branches on uncertain outcomes.

- **Sequential**: Each action changes the state and can affect which subsequent actions lead to the goal. The state representation retains the information needed to predict future transitions.
- **Single Agent**: There is no interaction with other agents that could introduce variability.
- **Fully Observable**: The agent has complete information about the environment, allowing for precise decision-making.
- **Deterministic**: The outcome of actions is predictable, with no randomness affecting the result.
- **Static**: The environment does not change independently while the agent plans; executing an action still changes the state.
- **Discrete**: States and actions are distinct alternatives that can be considered separately; the space need not be finite.

This formulation connects familiar graph-traversal algorithms with planning: the search selects a sequence in the model before the agent carries it out in the environment.

Search Problem Definition

- A collection of **states**, referred to as the **state space**.
- An **initial state** where the agent begins.
- One or more **goal states** that define successful outcomes, recognized by a **goal test**.
- A set of **actions** available in a given state s.
- A **transition model** that determines the next state based on the current state and selected action.
- An **action cost function** that specifies the cost of performing action a in state s to reach state s'.

Definitions

- A **path** is a sequence of states connected by actions; it can be recorded as the action sequence or the resulting state sequence.
- A **solution** is a path that connects the initial state to the goal state.
- An **optimal solution** is the path with the lowest cost among all possible solutions.

We assume that the **path cost** is the sum of the individual action costs, and all costs are positive. The state space can be conceptualized as a **graph**, where the nodes represent the states and the edges correspond to the actions.

In certain problems, multiple optimal solutions may exist. However, it is typically sufficient to identify and report a single optimal solution. Providing all optimal solutions can significantly increase time and space complexity for some problems.

Example: 8-Puzzle

In [1]:
```
import random
import matplotlib.pyplot as plt
import numpy as np

random.seed(58)

def is_solvable(tiles):
    # Count the inversions in the flattened list of tiles (excluding the bla
    # Assumption: board is square and width is odd.
    inversions = 0
    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] != 0 and tiles[j] != 0 and tiles[i] > tiles[j]:
                inversions += 1
    return inversions % 2 == 0

def generate_solvable_board():
    # Generate a random board configuration that is guaranteed to be solvabl
    tiles = list(range(9))
    random.shuffle(tiles)
    while not is_solvable(tiles):
        random.shuffle(tiles)

    return tuple(tiles)

def plot_board(board, title, num_pos, position):
    ax = plt.subplot(1, num_pos, position)
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])

    board = np.array(board).reshape(3, 3).tolist() # Reconfigure the grid to

    # Use a color map to display the numbers
    cmap = plt.cm.plasma
    norm = plt.Normalize(vmin=-1, vmax=8)

    for i in range(3):
        for j in range(3):
            tile_value = board[i][j]
            color = cmap(norm(tile_value))
            ax.add_patch(plt.Rectangle((j, 2 - i), 1, 1, facecolor=color, ec
            if tile_value == 0:
                ax.add_patch(plt.Rectangle((j, 2 - i), 1, 1, facecolor='whit
            else:
                ax.text(j + 0.5, 2 - i + 0.5, str(tile_value),
                        fontsize=16, ha='center', va='center', color='black'
```

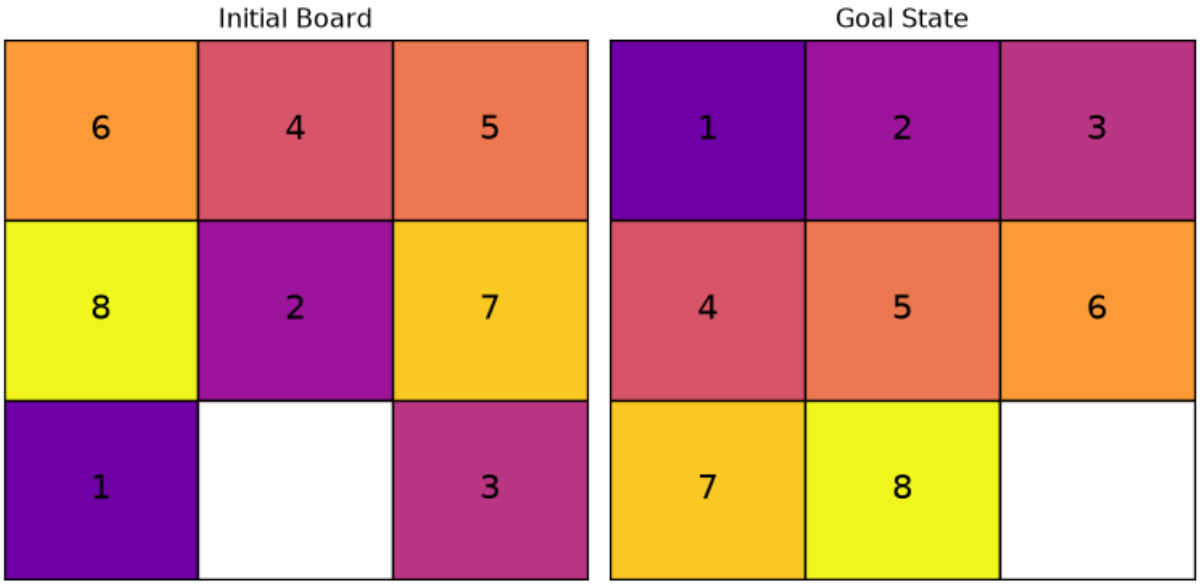
```
    ax.set_xlim(0, 3)
    ax.set_ylim(0, 3)
```

Example: 8-Puzzle

In [2]:
```
# Generate initial solvable board
initial_board = generate_solvable_board()

# Define goal state
goal_board = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

# Plot both boards
plt.figure(figsize=(8, 4))
plot_board(initial_board, "Initial Board", 2, 1)
plot_board(goal_board, "Goal State", 2, 2)
plt.tight_layout()
plt.show()
```

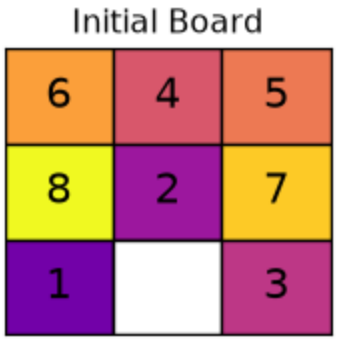


8-Puzzle

- How can the **states** be represented?
- What constitutes the **initial state**?
- What defines the **actions**?
- What would constitute a **path**?
- What characterizes the **goal state**?
- What would constitute a **solution**?

- What should be the **cost of an action**?

In [3]:
```
plt.figure(figsize=(2, 2))
plot_board(initial_board, "Initial Board", 1, 1)
plt.tight_layout()
plt.show()
```



- Each state is represented as a flat tuple containing the integers 0 to 8. Each number corresponds to a tile, and its position in the tuple reflects its location in the grid, with 0 denoting the blank space.
- The initial state is a permutation of the numbers 0 to 8.
- Actions move the **blank left**, **right**, **up**, or **down** by swapping it with an adjacent tile. These directions describe the blank's movement, not the numbered tile's movement.
- A path would be a sequence of actions, say **left**, **left**, **up**.
- The transition model maps a given state and action to a new state. Not all actions are feasible from every state; for instance, if the blank space is at the edge of the grid, only certain moves are possible, such as down, up, or left.
- The goal state is achieved when the tuple is ordered from 1 to 8, followed by 0, indicating that the tiles are arranged correctly. How many goal states are there? This formulation specifies one goal state, although another problem could permit several.
- A solution would be a valid path transforming an initial state into a goal state.
- Each action incurs a cost of 1.

How many possible states are there?

There are $9! = 362,880$ arrangements of the eight tiles and the blank. Only half, $9!/2 = 181,440$, are reachable from a fixed initial state. Exhaustive graph search is feasible at this scale.

How many states are there for the 15-Puzzle?

There are $16! = 20,922,789,888,000$ arrangements of the fifteen tiles and the blank, with $16!/2 = 10,461,394,944,000$ reachable from a fixed initial state: about 10.46 trillion. This much larger space motivates more selective search. See the 15-puzzle solver's state counts.

Are all the boards solvable?

In [4]:
```
some_board = (1, 2, 3,
              4, 5, 6,
              8, 7, 0)

plt.figure(figsize=(2, 2))
plot_board(some_board, "", 1, 1)
plt.tight_layout()
plt.show()
```



Are all the boards solvable?

- An **8-puzzle** board is solvable if it has the same **inversion parity** as the goal state.
- An **inversion** is a pair of tiles (excluding the blank) that are in the wrong order relative to each other, when reading the board as a one-dimensional sequence (left-to-right, top-to-bottom).
- When the goal is the ordered sequence from 1 to 8, there are no inversions, and therefore parity is even.
- See Archer (1999) or Slider Puzzle, Princeton

For an odd-width board such as the 3×3 puzzle, two arrangements are connected by legal moves exactly when their inversion parities agree. The simple inversion-only rule does not extend to even-width boards: the 4×4 puzzle also requires the blank's row in the solvability test.

`count_even_odd_parity`

In [5]:
```python
from itertools import permutations

def count_even_odd_parity():
    even_count = 0
    odd_count = 0

    for perm in permutations(range(9)):  # 0 represents the blank
        inversions = 0
        for i in range(9):
            for j in range(i + 1, 9):
                if perm[i] != 0 and perm[j] != 0 and perm[i] > perm[j]:
                    inversions += 1
        if inversions % 2 == 0:
            even_count += 1
        else:
            odd_count += 1

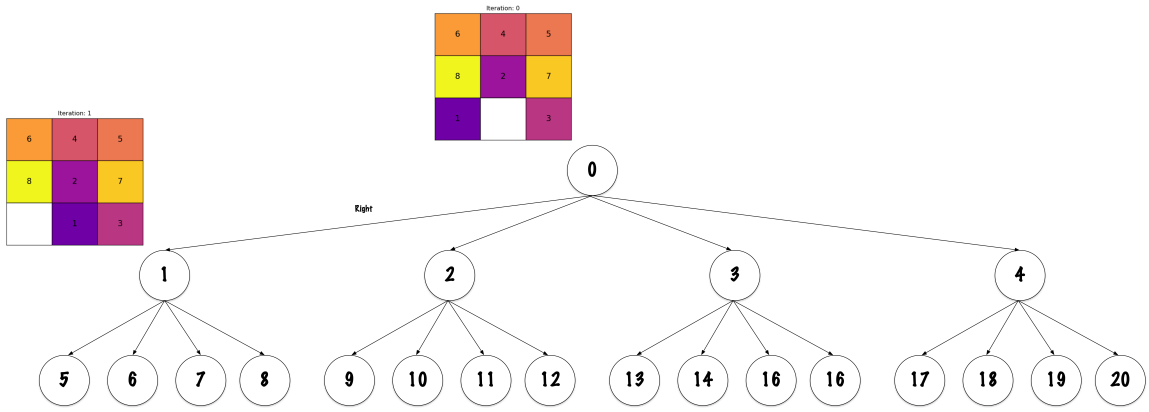
    return even_count, odd_count
```

In [6]:
```python
even, odd = count_even_odd_parity()

print(f"Even parity: {even}")
print(f"Odd parity: {odd}")
```

```
Even parity: 181440
Odd parity: 181440
```

Search Tree

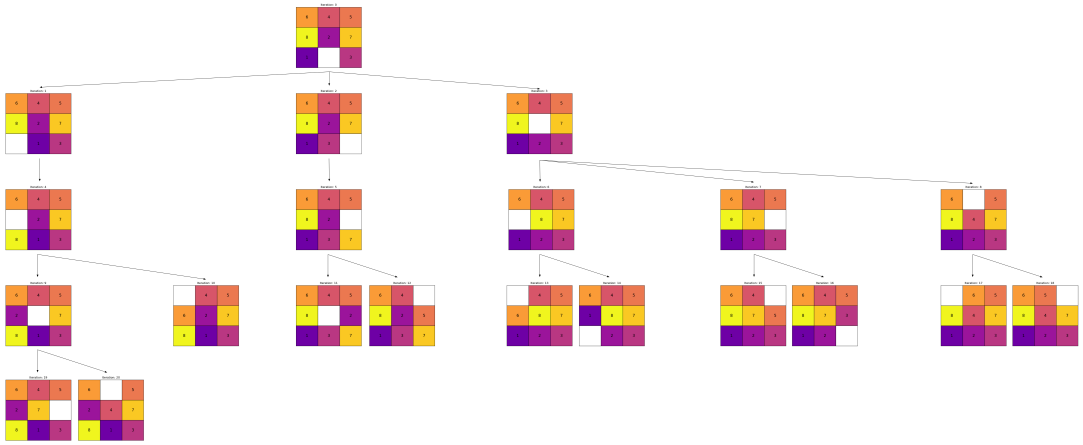


A **search tree** records possible **paths** from the **initial state**. Each **node** records a state reached along a particular path, and each **edge** represents an action. Different nodes can represent the same state.

The search algorithms construct nodes representing states reached by particular paths. A node may retain its parent, the action used to reach it, and the path cost. In the implementations below, a pair `(state, path)` stores the state and the preceding state sequence.

The **state-space graph** describes the problem: its vertices are states and its edges are actions. The **search tree** describes paths considered during search. Several paths can reach the same state, so the same state can occur at several tree nodes. The algorithms below use repeated-state detection, making them **graph-search** implementations; they do not retain every repeated occurrence in the conceptual tree.

Search Tree



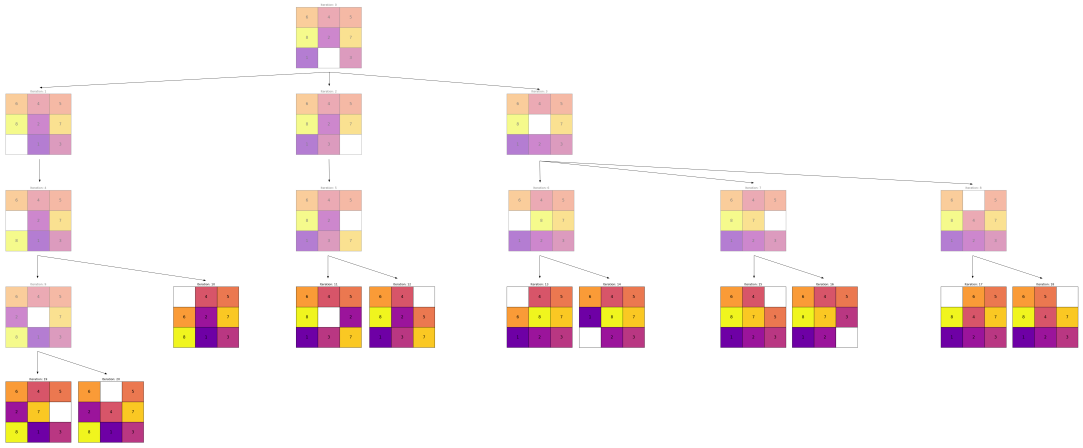
An example of a **search tree** for the **8-Puzzle**. The solution here is incomplete.

Search Tree

- The **root** of the search tree represents the **initial state** of the problem.
- **Expanding** a node generates successor states by applying the **available actions** to its state.
- The **result** of an action is the new state achieved after **applying that action** to the **current state**.
- Every node except the **root** has a **parent**, including **leaf** nodes. Expanding a node may generate **children**.

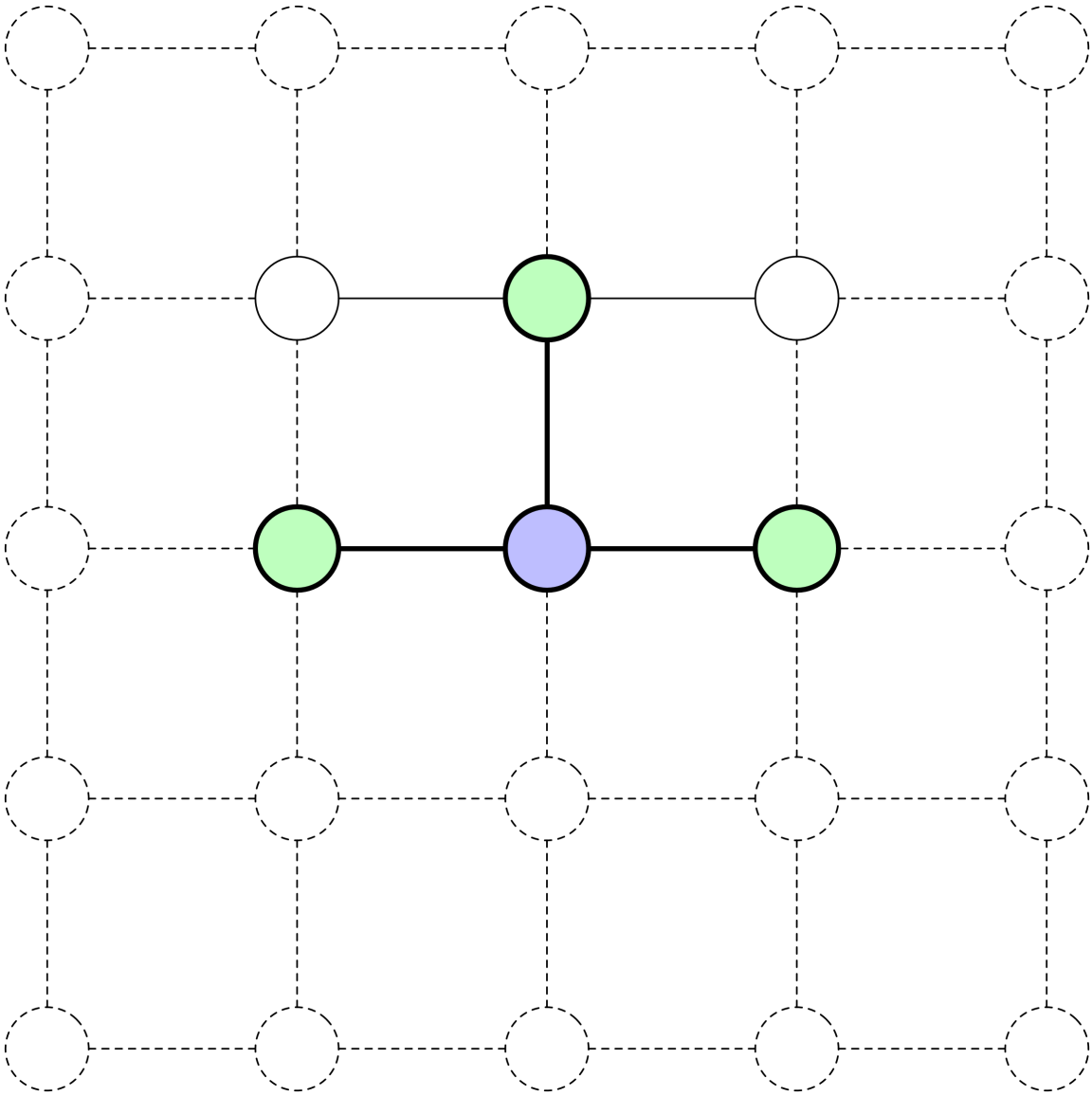
A distinctive feature of a search algorithm is its method for **selecting the next node to expand**.

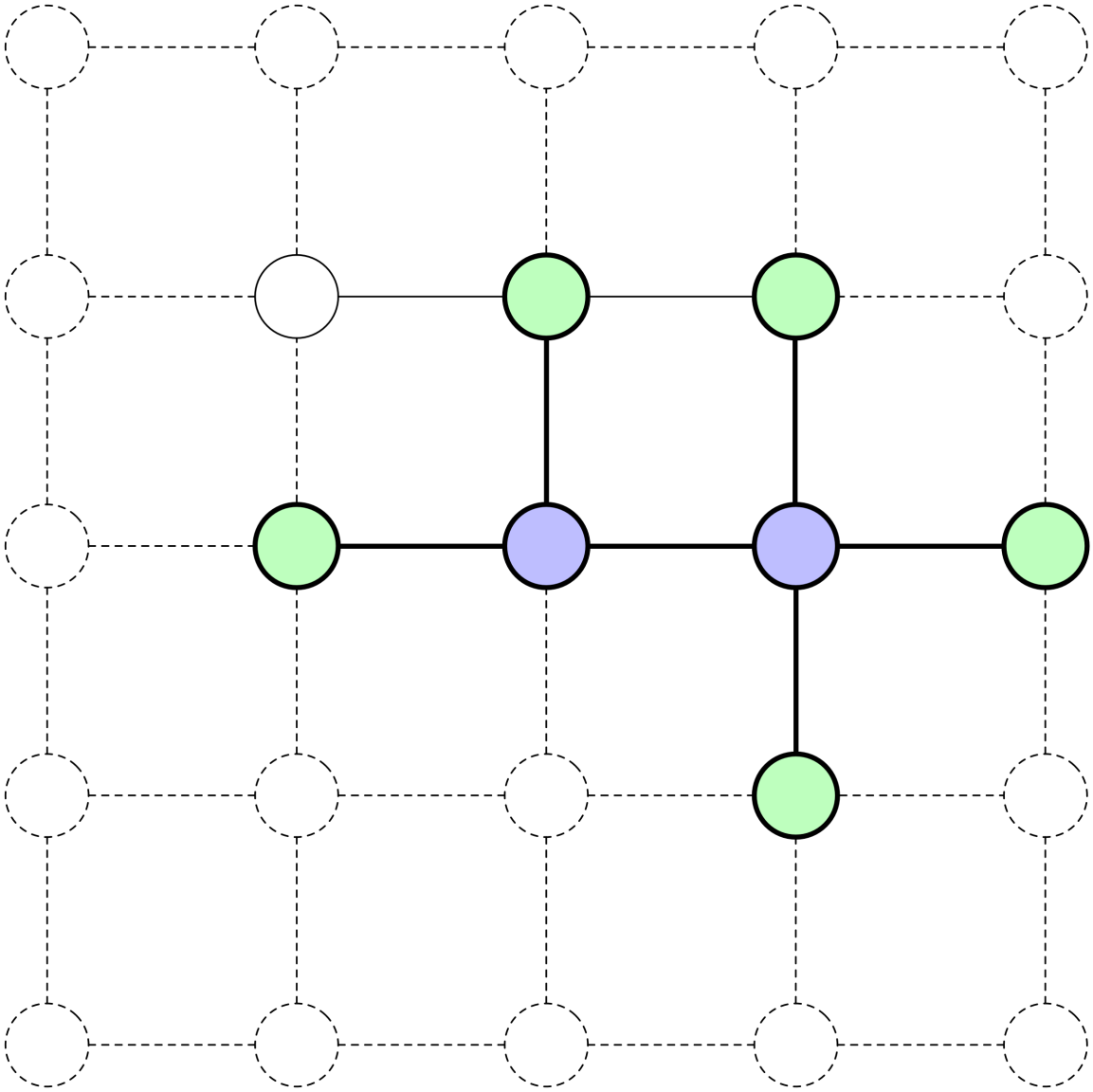
Frontier

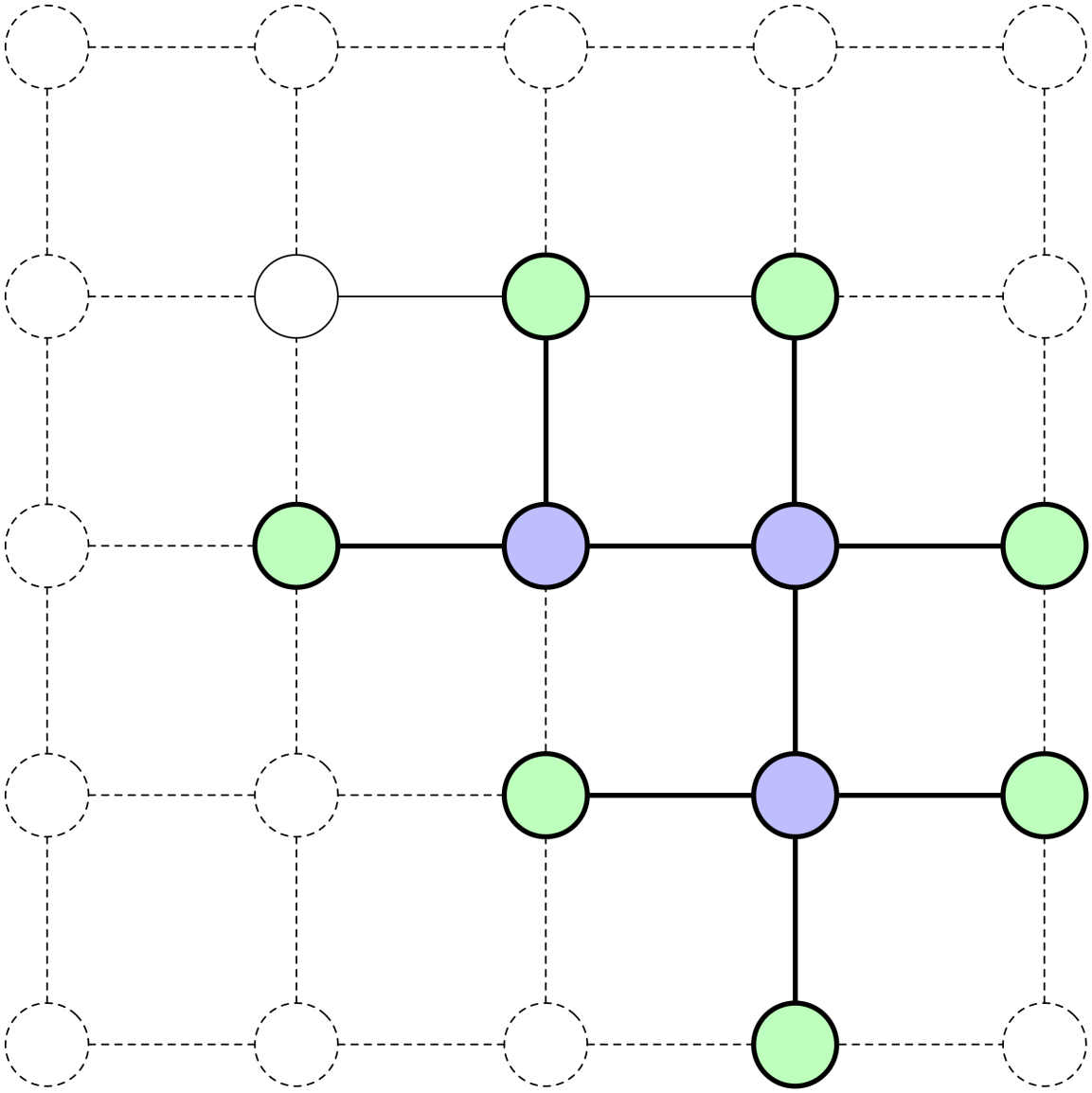


Any state corresponding to a node in the search tree is considered **reached**. **Frontier** nodes are those that have been **reached** but have **not yet been expanded**. Above, there are **10 expanded nodes** and **11 frontier nodes**, resulting in a total of 21 nodes that have been **reached**.

Frontier



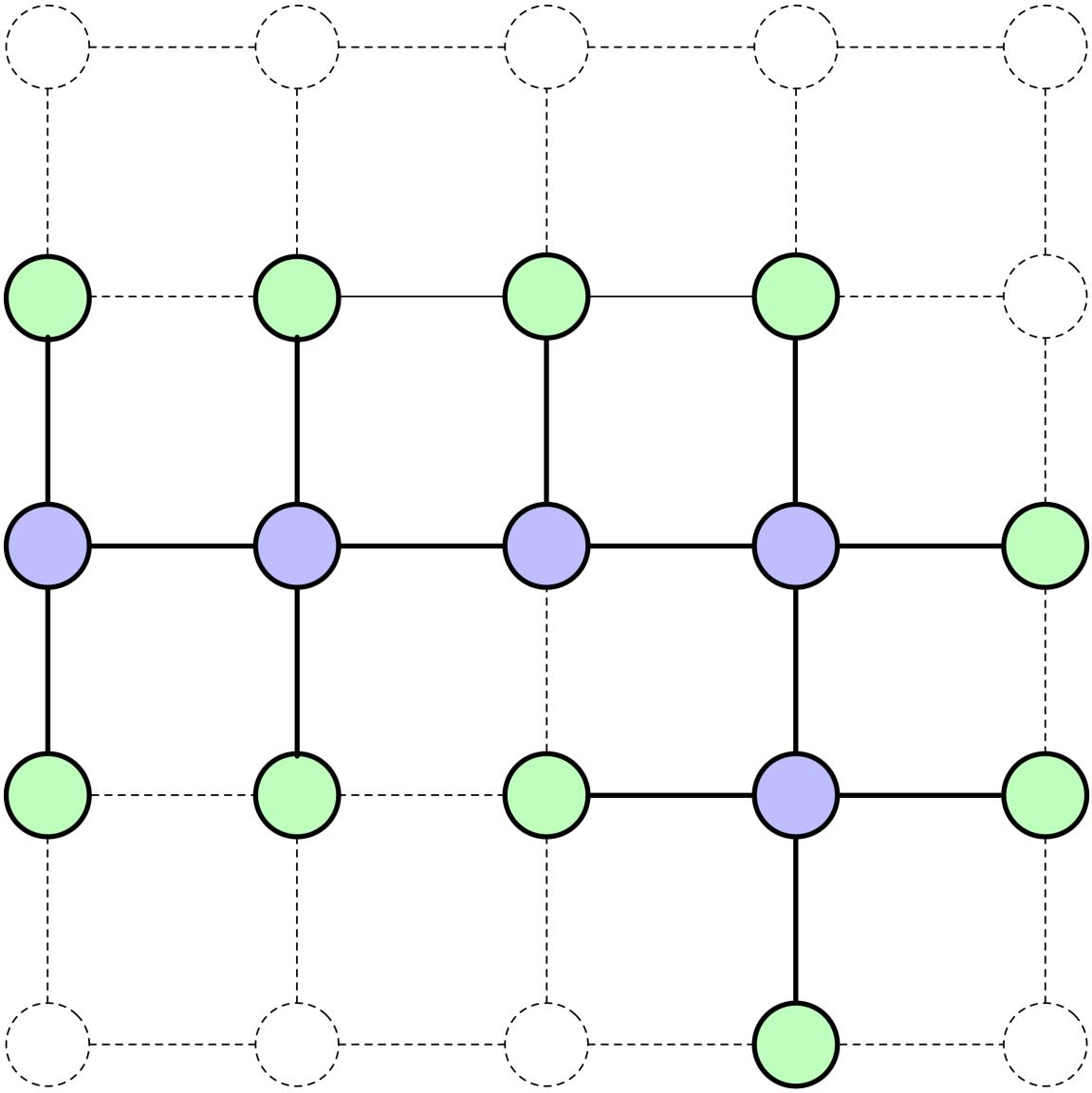


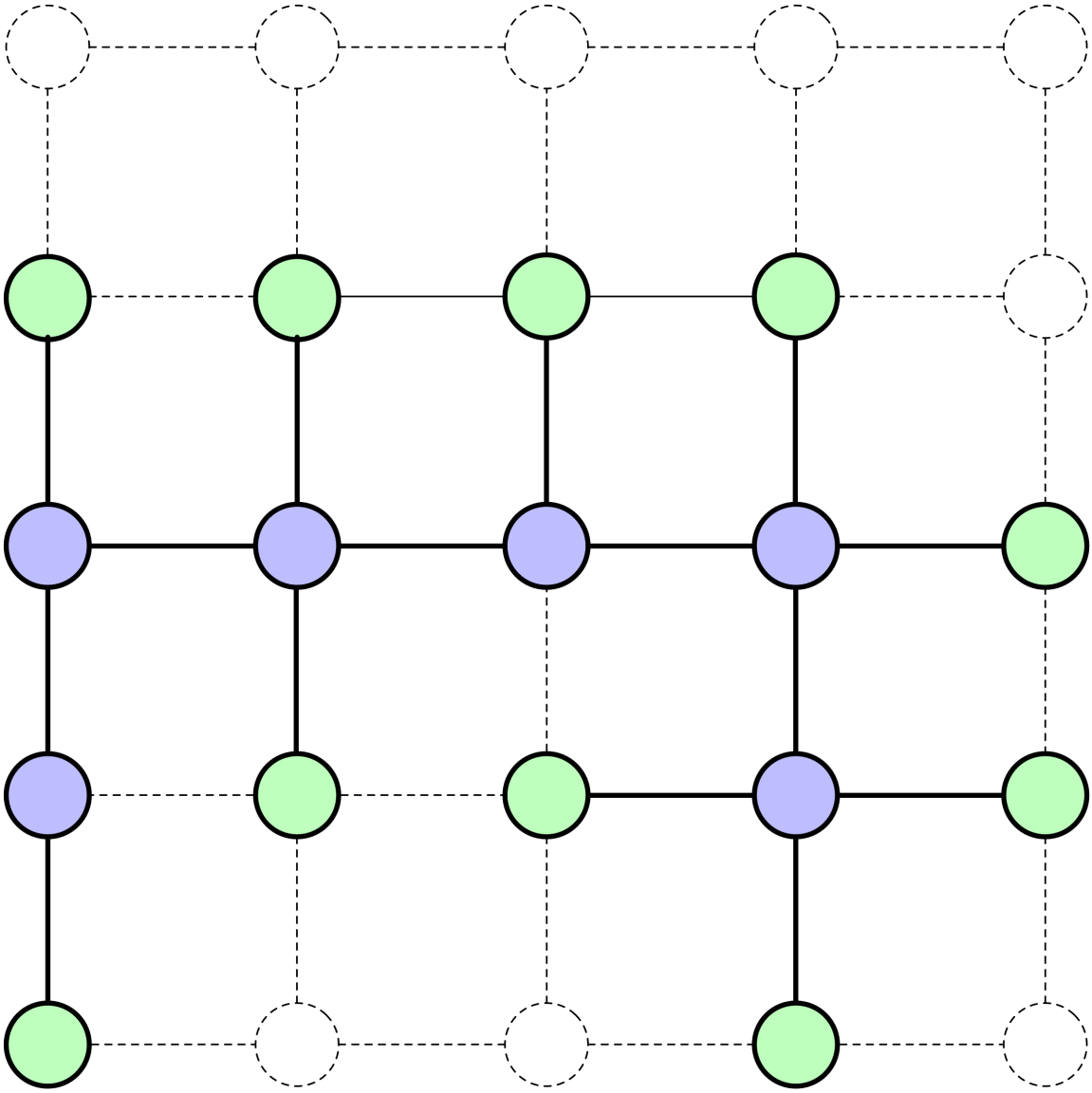


In the 8-Puzzle, the blank can move **left**, **right**, **up**, or **down** when the board boundaries permit. The diagrams schematically show the search frontier: **purple nodes** are expanded; **green nodes** are reached but not expanded. They are not the physical puzzle board.

These schematic graphs illustrate how a frontier changes as nodes are expanded. Expanding a green node generates its successors: newly reached states enter the frontier, while already reached states are not added again in the graph-search implementations below. Purple nodes have already been expanded. The drawn grid positions do not encode 8-puzzle tiles or determine which puzzle moves are legal.

Frontier





Uninformed search

Definition

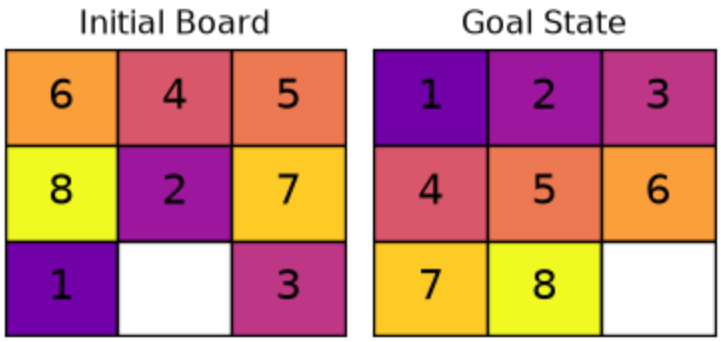
An **uninformed search** (or blind search) uses the information in the problem definition without an additional **heuristic estimate of the remaining cost to a goal**. Problem-specific actions, transition rules, goal tests, and action costs are still available.

State Representation

In [7]:
```
# Plot both boards
initial_state_8 = (6, 4, 5,
                   8, 2, 7,
                   1, 0, 3)
goal_state_8 = (1, 2, 3,
```

```
                4, 5, 6,
                7, 8, 0)

plt.figure(figsize=(4, 2))
plot_board(initial_state_8, "Initial Board", 2, 1)
plot_board(goal_state_8, "Goal State", 2, 2)
plt.tight_layout()
plt.show()
```



In [8]:
```
initial_state_8 = (6, 4, 5,
                   8, 2, 7,
                   1, 0, 3)

goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)
```

States are flat **tuples of integers**; `0` represents the **blank tile**. The `reached` set requires **hashable** states: tuples of integers are immutable and hashable, whereas mutable lists are not.

`is_goal`

In [9]:
```
def is_goal(state, goal_state):
    """Determines if a given state matches the goal state."""
    return state == goal_state
```

Auxiliary function implementing the goal test.

`expand`

In [10]:
```
def expand(state):
    """Generates successor states by moving the blank tile in all possible d
    size = int(len(state) ** 0.5)  # Determine puzzle size (3 for 8-puzzle,
    idx = state.index(0)  # Find the index of the blank tile represented by
    x, y = idx % size, idx // size  # Convert index to (x, y) coordinates
    neighbors = []

    # Define possible moves: Left, Right, Up, Down
```

```
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        # Check if the new position is within the puzzle boundaries
        if 0 <= nx < size and 0 <= ny < size:
            n_idx = ny * size + nx
            new_state = list(state)
            # Swap the blank tile with the adjacent tile
            new_state[idx], new_state[n_idx] = new_state[n_idx], new_state[i
            neighbors.append(tuple(new_state))
    return neighbors
```

expand

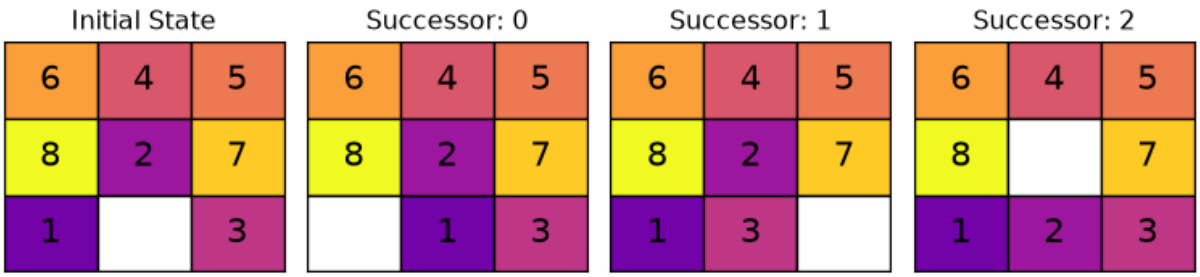
In [11]:
```
plt.figure(figsize=(8, 2))

successors = expand(initial_state_8)

plot_board(initial_state_8, "Initial State", 4, 1)

for i, successor in enumerate(successors):
    plot_board(successor, f"Successor: {i}", 4, i+2)

plt.tight_layout()
plt.show()
```



In [12]:
```
expand(initial_state_8)
```

```
[(6, 4, 5, 8, 2, 7, 0, 1, 3),
 (6, 4, 5, 8, 2, 7, 1, 3, 0),
 (6, 4, 5, 8, 0, 7, 1, 2, 3)]
```

is_empty

In [13]:
```
def is_empty(frontier):
    """Checks if the frontier is empty."""
    return len(frontier) == 0
```

If the frontier becomes empty (no more nodes to be expanded), the problem has no solution.

Are there 8-Puzzle boards that have no solutions?

The solvability of the 8-puzzle depends on the number of **inversions** in the initial state. An inversion is a pair of tiles where a higher-numbered tile precedes a lower-numbered tile when the puzzle is viewed as a sequence (excluding the blank tile).

If the initial state and the goal state do not have the same inversion parity, then the board has no solution.

`print_solution`

In [14]:
```python
def print_solution(solution):
    """Prints the sequence of steps from the initial to the goal state."""
    size = int(len(solution[0]) ** 0.5)
    for step, state in enumerate(solution):
        print(f"Step {step}:")
        for i in range(size):
            row = state[i*size:(i+1)*size]
            print(' '.join(str(n) if n != 0 else ' ' for n in row))
        print()
```

Cycles

. . .

A path that revisits the same states forms a **cycle**.

Allowing cycles would render the resulting **search tree infinite**.

To prevent this, we monitor the states that have been reached, though this incurs a **memory cost**.

Breadth-first search

Breadth-first search

In [15]:
```python
from collections import deque
```

Breadth-first search (BFS) employs a **queue** to manage the frontier nodes, which are also known as the open list.

Breadth-first search

In [16]:
```python
def bfs(initial_state, goal_state):

    frontier = deque()  # Initialize the queue for BFS
```

```
    frontier.append((initial_state, []))  # A node: (state, preceding states

    reached = {initial_state}

    expanded = 0  # Count non-goal nodes whose successors are generated

    while not is_empty(frontier):
        current_state, path = frontier.popleft()

        if is_goal(current_state, goal_state):
            print(f"Nodes expanded: {expanded}")
            return path + [current_state]  # Return the successful path

        expanded += 1

        for neighbor in expand(current_state):
            if neighbor not in reached:
                reached.add(neighbor)
                frontier.append((neighbor, path + [current_state]))

    return None  # No solution found
```

Find a path with the **fewest actions**. This is a **least-cost solution when all action costs are equal**, as in the 8-puzzle.

BFS selects nodes in nondecreasing depth because the frontier is a first-in, first-out queue. With equal action costs, the first path that reaches a state is a least-cost path to that state, so later occurrences can be discarded. Unequal action costs break the equivalence between fewest actions and least cost.

The `reached` set contains states in the frontier as well as expanded states. The `expanded` counter increases only when a non-goal node's successors are generated; selecting a goal does not increase it. The returned solution is a sequence of states, including both endpoints, so its number of moves is `len(solution) - 1`.

Simple Case

In [17]:
```
plt.figure(figsize=(8, 2))

initial_state_8 = (1, 2, 3,
                   4, 0, 6,
                   7, 5, 8)
goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)

solutions = bfs(initial_state_8, goal_state_8)

for i, solution in enumerate(solutions):
    plot_board(solution, f"Step: {i}", 3, i+1)
```

```
plt.tight_layout()
plt.show()
```

```
Nodes expanded: 12
```



In [18]:
```
initial_state_8 = (1, 2, 3,
                   4, 0, 6,
                   7, 5, 8)
goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)

bfs(initial_state_8, goal_state_8)
```

```
Nodes expanded: 12
[(1, 2, 3, 4, 0, 6, 7, 5, 8),
 (1, 2, 3, 4, 5, 6, 7, 0, 8),
 (1, 2, 3, 4, 5, 6, 7, 8, 0)]
```

Challenging Case

In [19]:
```
initial_state_8 = (6, 4, 5,
                   8, 2, 7,
                   1, 0, 3)
goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)

print("Solving 8-puzzle with BFS...")

solution_8_bfs = bfs(initial_state_8, goal_state_8)

if solution_8_bfs:
    print(f"BFS Solution found in {len(solution_8_bfs) - 1} moves:")
    print_solution(solution_8_bfs)
else:
    print("No solution found for 8-puzzle using BFS.")
```

```
Solving 8-puzzle with BFS...
Nodes expanded: 145605
BFS Solution found in 25 moves:
Step 0:
6 4 5
8 2 7
1   3

Step 1:
6 4 5
8 2 7
  1 3

Step 2:
6 4 5
  2 7
8 1 3

Step 3:
6 4 5
2   7
8 1 3

Step 4:
6   5
2 4 7
8 1 3

Step 5:
  6 5
2 4 7
8 1 3

Step 6:
2 6 5
  4 7
8 1 3

Step 7:
2 6 5
4   7
8 1 3

Step 8:
2 6 5
4 1 7
8   3

Step 9:
2 6 5
4 1 7
  8 3

Step 10:
2 6 5
  1 7
```

```
4 8 3

Step 11:
2 6 5
1   7
4 8 3

Step 12:
2 6 5
1 7
4 8 3

Step 13:
2 6 5
1 7 3
4 8

Step 14:
2 6 5
1 7 3
4   8

Step 15:
2 6 5
1   3
4 7 8

Step 16:
2   5
1 6 3
4 7 8

Step 17:
2 5
1 6 3
4 7 8

Step 18:
2 5 3
1 6
4 7 8

Step 19:
2 5 3
1   6
4 7 8

Step 20:
2   3
1 5 6
4 7 8

Step 21:
  2 3
1 5 6
4 7 8
```

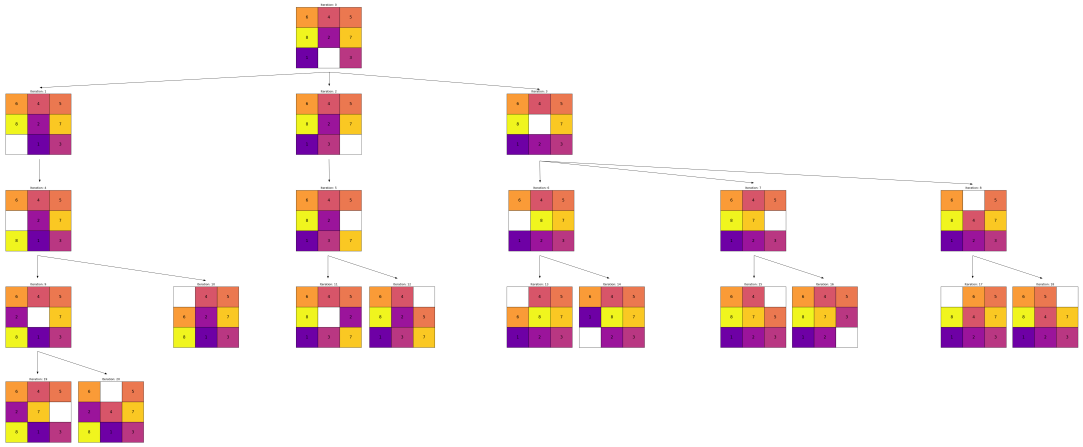
```
Step 22:
1 2 3
  5 6
4 7 8

Step 23:
1 2 3
4 5 6
  7 8

Step 24:
1 2 3
4 5 6
7   8

Step 25:
1 2 3
4 5 6
7 8
```

BFS Search Tree



The search tree above illustrates the first 20 node expansions of breadth-first search (BFS) for the specified initial state.

Depth-First Search

Depth-First Search

In [20]:
```python
def dfs(initial_state, goal_state):

    frontier = [(initial_state, [])]  # A node: (state, preceding states)

    reached = {initial_state}

    expanded = 0
```

```
    while not is_empty(frontier):
        current_state, path = frontier.pop()

        if is_goal(current_state, goal_state):
            print(f"Nodes expanded: {expanded}")
            return path + [current_state]  # Return the successful path

        expanded += 1

        for neighbor in expand(current_state):
            if neighbor not in reached:
                reached.add(neighbor)
                frontier.append((neighbor, path + [current_state]))

    return None  # No solution found
```

What is the behaviour of **depth-first search (DFS)**?

Depth-first search (DFS) uses a last-in, first-out **stack**. Newly generated nodes are selected before older alternatives, so the search follows a branch deeply before returning to other frontier nodes. The successor order matters: because `expand` lists left, right, up, down and the stack removes the last entry first, DFS tries the last newly added successor first.

When does the deepening process halt?

It stops following a branch when a goal is found or no new successor can be added, because there are no available actions or all successors have already been reached.

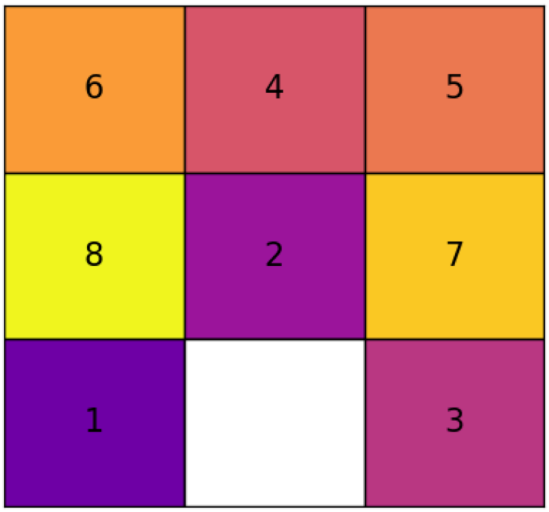
What happens next?

The algorithm pops the most recently added remaining frontier node. The stored path records how that alternative was reached; the code does not undo moves on a shared board.

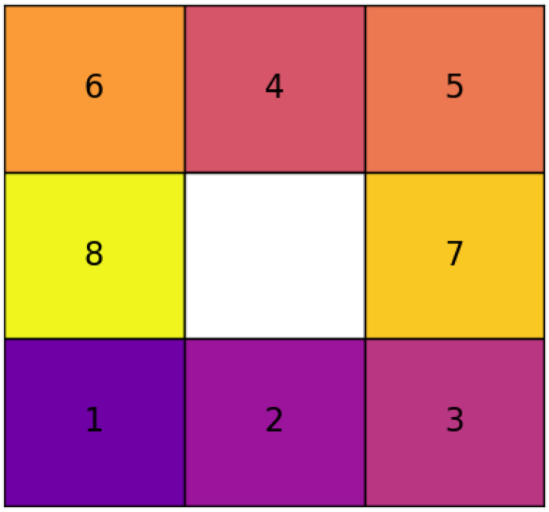
If that node also has no new successors, the algorithm continues with the next stack entry. Repeated-state detection prevents cycles from continually adding the same states.

DFS Search Tree

Iteration: 0

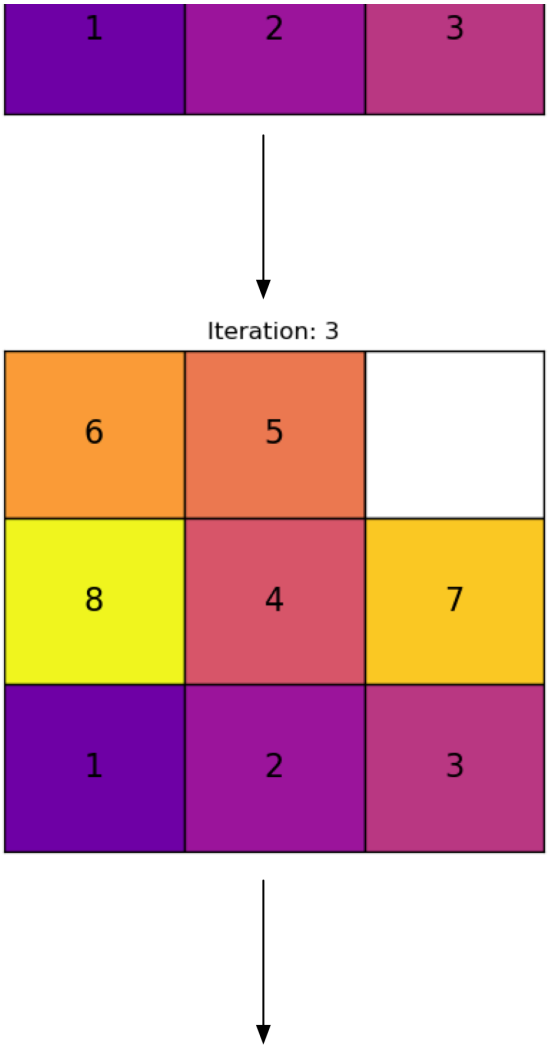


Iteration: 1

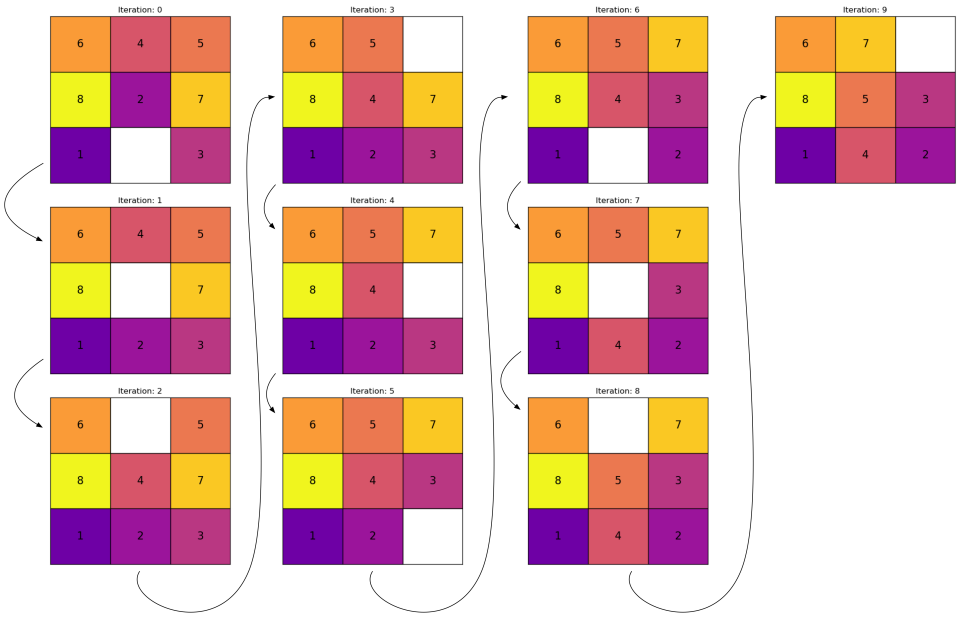


Iteration: 2





DFS Search Tree



Simple Case

In [21]:
```
plt.figure(figsize=(8, 2))

initial_state_8 = (1, 2, 3,
                   4, 0, 6,
                   7, 5, 8)
goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)

solutions = dfs(initial_state_8, goal_state_8)

for i, solution in enumerate(solutions):
    plot_board(solution, f"Step: {i}", 3, i+1)

plt.tight_layout()
plt.show()
```

```
Nodes expanded: 2
```



In [22]:
```
initial_state_8 = (1, 2, 3,
                   4, 0, 6,
                   7, 5, 8)
goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)

dfs(initial_state_8, goal_state_8)
```

```
Nodes expanded: 2
[(1, 2, 3, 4, 0, 6, 7, 5, 8),
 (1, 2, 3, 4, 5, 6, 7, 0, 8),
 (1, 2, 3, 4, 5, 6, 7, 8, 0)]
```

Challenging Case

In [23]:
```
initial_state_8 = (6, 4, 5,
                   8, 2, 7,
                   1, 0, 3)
goal_state_8 = (1, 2, 3,
                4, 5, 6,
                7, 8, 0)

print("Solving 8-puzzle with DFS...")

solution_8_dfs = dfs(initial_state_8, goal_state_8)
```

```python
if solution_8_dfs:
    print(f"DFS Solution found in {len(solution_8_dfs) - 1} moves:")
    # print_solution(solution_8_dfs)
else:
    print("No solution found for 8-puzzle using DFS.")
```

```
Solving 8-puzzle with DFS...
Nodes expanded: 1187
DFS Solution found in 1157 moves:
```

Remarks

- **BFS:** 25 moves (optimal); 145,605 non-goal nodes expanded.
- **DFS:** 1,157 moves; 1,187 non-goal nodes expanded.

Property	BFS	DFS
Frontier	FIFO queue	LIFO stack
Complete on finite reachable graphs	Yes	Yes
Fewest actions / least cost (equal action costs)	Guaranteed	Not guaranteed
Memory retained	Frontier, reached set, copied paths	Frontier, reached set, copied paths

How can solutions be discovered **more efficiently**?

Will **Depth-First Search (DFS)** invariably yield sub-optimal solutions?

No. DFS can return an optimal solution if it encounters one first, but it provides no guarantee that the first solution has the fewest actions or lowest cost.

Is it possible for DFS to discover solutions superior to the optimal solution?

Certainly not; such solutions would either be invalid (involving impossible moves) or indicate an error in your estimation.

Does this imply that depth-first search (DFS) has no practical applications?

When is it appropriate to use DFS?

Breadth-first search (BFS) expands its frontier systematically in all directions, leading to rapid growth in memory requirements.

DFS often has a smaller frontier because it follows one branch before trying alternatives. However, the code shown here retains every reached state and copies paths into frontier entries. Its total memory is therefore not limited to the active branch, and a small frontier alone does not establish low total memory usage.

The usual small-memory description of DFS applies to implementations that retain the active path and pending alternatives, without a global record of all reached states. Such backtracking implementations can be useful when systematically exploring possible solutions. Their duplicate-handling and termination properties differ from those of the graph-search code above.

The programming language Prolog uses depth-first backtracking to search for answers. It can enumerate answers, but an infinite branch can prevent it from reaching other answers.

Completeness means finding a solution whenever one exists. Both graph-search implementations shown here are complete on a finite reachable state space because each state is added at most once. BFS also finds a finite-depth solution with finite branching in an infinite space; DFS can instead follow an infinite branch indefinitely.

These numerical results apply to the specified initial board and successor order. Expansion counts describe search effort, not elapsed time or peak memory; different boards or move orders can change the comparison substantially.

Depth-limited and iterative deepening search would be alternative uninformed search algorithms.

Heuristic functions use additional knowledge to estimate the remaining cost to a goal and guide search. They are the focus of the next lecture.

Epilogue

Summary

- Justification for Studying Search
- Key Terminology and Concepts
- Uninformed Search Algorithms
    - Breadth-First Search (BFS)
    - Depth-First Search (DFS)
- Implementations: completeness, cost optimality, and memory
- **Justification for Studying Search:**
    - Emphasized the shift from solely focusing on machine learning to incorporating search algorithms.
    - Highlighted the role of search in advanced AI systems like AlphaGo, AlphaZero, and MuZero.

    - Noted that search algorithms are crucial for planning, reasoning, and will be increasingly significant.
- **Historical Timeline of Search Algorithms:**
    - Selected milestones range from A* in 1968 to FunSearch in 2023 and AlphaGeometry in 2024.
    - The timeline includes explicit search methods, reinforcement-learning advances, and systems combining learning with search.
- **Applications of Search:**
    - **Pathfinding and Navigation:** Finding optimal paths in robotics and games.
    - **Puzzle Solving:** Solving problems like the 8-puzzle and Sudoku.
    - **Network Analysis:** Analyzing connectivity and shortest paths in networks.
    - **Game Playing:** Evaluating moves in games like chess or Go.
    - **Scheduling and Resource Allocation:** Planning tasks and allocating resources efficiently.
    - **Configuration Problems:** Assembling components to meet specific requirements.
    - **Decision Making under Uncertainty:** Making decisions in dynamic and uncertain environments.
    - **Storytelling:** Guiding language models with plans from automated planners.
- **Key Terminology and Concepts:**
    - **Agent:** An entity that performs actions to achieve goals.
    - **Environment Characteristics:** Fully observable, single-agent, sequential, deterministic, static, and discrete environments were focused on.
    - **Search Problem Definition:**
        - **State Space:** All possible states.
        - **Initial State:** Where the agent starts.
        - **Goal State(s):** Desired outcome(s).
        - **Actions:** Possible moves from a state.
        - **Transition Model:** Rules determining state changes.
        - **Action Cost Function:** Cost associated with actions.
- **Uninformed Search Algorithms:**
    - **Breadth-First Search (BFS):**
        - Explores the search space level by level.
        - Guarantees the fewest actions, and least cost when action costs are equal, but can be memory-intensive.
        - Implemented using a queue.
    - **Depth-First Search (DFS):**
        - Explores as deep as possible along each branch before backtracking.
        - Does not guarantee the fewest actions or least cost. A smaller frontier does not remove the memory cost of the global reached set and copied paths in

this implementation.
    - Implemented using a stack.
- **Implementing Uninformed Search:**
    - Used the **8-Puzzle** as an example problem.
    - Represented states as flat tuples of integers, with `0` as the blank tile.
    - Demonstrated BFS and DFS implementations in Python.
    - On the specified board and move order, BFS found the optimal solution after more expansions, while DFS found a longer solution after fewer expansions.
- **Completeness and Repeated States:**
    - Both implementations find a solution when one exists in a finite reachable state space.
    - `reached` includes frontier and expanded states; `expanded` counts non-goal nodes whose successors are generated.
- **Limitations of Uninformed Search:**
    - Inefficient for large or complex problems due to exhaustive nature.
    - Without an estimate of remaining goal cost, they can explore many states that do not help reach the goal.

Next lecture

- We will further explore heuristic functions and examine additional search algorithms.

References

Archer, Aaron F. 1999. "A Modern Treatment of the 15 Puzzle." *The American Mathematical Monthly* 106 (9): 793–99. https://doi.org/10.1080/00029890.1999.12005124.

Hart, Peter E., Nils J. Nilsson, and Bertram Raphael. 1968. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics* 4 (2): 100–107. https://doi.org/10.1109/tssc.1968.300136.

Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson. http://aima.cs.berkeley.edu/.

Schrittwieser, Julian, Ioannis Antonoglou, Thomas Hubert, et al. 2020. "Mastering Atari, Go, chess and shogi by planning with a learned model." *Nature* 588 (7839): 604–9. https://doi.org/10.1038/s41586-020-03051-4.

Silver, David, Aja Huang, Chris J. Maddison, et al. 2016. "Mastering the game of Go with deep neural networks and tree search." *Nature* 529 (7587): 484–89. https://doi.org/10.1038/nature16961.

Simon, Nisha, and Christian Muise. 2024. “Want To Choose Your Own Adventure? Then First Make a Plan.” *Proceedings of the Canadian Conference on Artificial Intelligence*.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

School of Electrical Engineering and **Computer Science** (EE**CS**)

University of Ottawa